

Lab 7: Python-to-Real Transfer

EE0849 Introduction to Robotics

Basri Erdogan

Table of contents

0.1	Objectives	2
1	Part 1: Python Preparation (25 minutes)	3
1.1	1.1 Build the FANUC Model	3
1.2	1.2 Define 5 Target Poses	3
1.3	1.3 Solve IK for Each Target	3
1.4	1.4 Exercise 1: Record Your Python Results	4
2	Part 2: Transfer to FANUC (20 minutes)	5
2.1	2.1 Enter Joint Angles as Position Registers	5
2.2	2.2 Create a Test Program	6
2.3	2.3 Safety Check Before Running	6
3	Part 3: Measurement (25 minutes)	7
3.1	3.1 Run the Program and Record Positions	7
3.2	3.2 Exercise 2: Complete the Measurement Table	7
3.3	3.3 Exercise 3: Repeat Measurement (Repeatability Check)	7
4	Part 4: Error Analysis (20 minutes)	8
4.1	4.1 Compute Position Errors	8
4.2	4.2 Exercise 4: Error Summary Table	8
4.3	4.3 Visualize the Errors	8
4.4	4.4 Exercise 5: Error Pattern Analysis	9
5	Part 5: Improving the Match (10 minutes)	10
5.1	5.1 Quick Calibration Check	10
5.2	5.2 Exercise 6: Discussion	10
6	Part 6: User Frame Portability (Extension)	11
6.1	6.1 Setup: Teach Positions in a User Frame	11
6.2	6.2 Move the Fixture	11
6.3	6.3 Re-calibrate and Verify	12
6.4	6.4 Analysis	12
7	Summary	13
7.1	7.1 Key Takeaways	13

Safety Reminder

- Start in **T1 mode** — max 250 mm/s
- Hold the **Deadman Switch** at all times
- Always test new programs in **single-step** mode first
- Know the **E-Stop** locations before starting

Prerequisites

This lab builds on Labs 5 & 6 (FANUC hardware) and Lab 4 (Numerical IK in Python). You should be comfortable with jogging, teaching positions, Position Registers, User Frames, and running TP programs.

0.1 Objectives

In this lab, you will:

1. Generate target poses and compute joint angles in Python
2. Transfer joint angles from Python to the FANUC Position Registers
3. Move the physical robot to each calculated position
4. Record the actual TCP position reported by the FANUC controller
5. Analyze the error between simulation prediction and physical reality
6. Experience User Frame portability by moving a fixture and re-calibrating

1 Part 1: Python Preparation (25 minutes)

1.1 1.1 Build the FANUC Model

Open your FANUC model from Lab 2 (or create it fresh):

```
import roboticstoolbox as rtb
import numpy as np
from spatialmath import SE3

# FANUC ER-4iA DH parameters (from datasheet)
robot = rtb.DHRobot([
    rtb.RevoluteDH(d=0.330, a=0.050, alpha=-np.pi/2),
    rtb.RevoluteDH(d=0, a=0.330, alpha=0),
    rtb.RevoluteDH(d=0, a=0.035, alpha=-np.pi/2),
    rtb.RevoluteDH(d=0.335, a=0, alpha=np.pi/2),
    rtb.RevoluteDH(d=0, a=0, alpha=-np.pi/2),
    rtb.RevoluteDH(d=0.080, a=0, alpha=0),
], name="FANUC_ER4iA")

print(robot)
```

Verify the model prints 6 joints (RRRRRR).

1.2 1.2 Define 5 Target Poses

Choose 5 points that are safely within the FANUC workspace:

```
# Define target TCP positions (in meters)
# Choose points at different distances and directions from base
targets = [
    SE3.Trans(0.400, 0.000, 0.300) * SE3.RPY([180, 0, 0], unit='deg'),
    SE3.Trans(0.350, 0.100, 0.250) * SE3.RPY([180, 0, 0], unit='deg'),
    SE3.Trans(0.300, -0.100, 0.350) * SE3.RPY([180, 0, 0], unit='deg'),
    SE3.Trans(0.450, 0.050, 0.200) * SE3.RPY([180, 0, 0], unit='deg'),
    SE3.Trans(0.400, -0.050, 0.400) * SE3.RPY([180, 0, 0], unit='deg'),
]
```

Safety Note: All target positions use a downward-pointing tool orientation (RPY = [180, 0, 0]) for safe operation. Do not use positions near joint limits or the workspace boundary.

1.3 1.3 Solve IK for Each Target

```
results = []

for i, T_target in enumerate(targets):
    sol = robot.ikine_LM(T_target)

    if sol.success:
        q_rad = sol.q
        q_deg = np.rad2deg(q_rad)

        # Verify with FK
        T_check = robot.fkine(q_rad)
```

```

pos_expected = T_check.t * 1000 # Convert to mm

results.append({
    'point': i + 1,
    'q_deg': q_deg,
    'pos_mm': pos_expected,
})

print(f"Point {i+1}:")
print(f"  Joint angles (deg): {q_deg.round(2)}")
print(f"  Expected TCP (mm): {pos_expected.round(1)}")
else:
    print(f"Point {i+1}: IK FAILED - choose a different target")

```

1.4 1.4 Exercise 1: Record Your Python Results

Fill in this table with your Python calculations:

Point	J1 (deg)	J2 (deg)	J3 (deg)	J4 (deg)	J5 (deg)	J6 (deg)
1						
2						
3						
4						
5						

Point	Expected X (mm)	Expected Y (mm)	Expected Z (mm)
1			
2			
3			
4			
5			

2 Part 2: Transfer to FANUC (20 minutes)

2.1 2.1 Enter Joint Angles as Position Registers

On the teach pendant:

1. Navigate to: **DATA** $\square$ **Position Registers**
2. Select **PR[1]** $\square$ **DETAIL**
3. Set Representation to **JOINT**
4. Enter the joint angles from your Python output for Point 1
5. Repeat for PR[2] through PR[5]

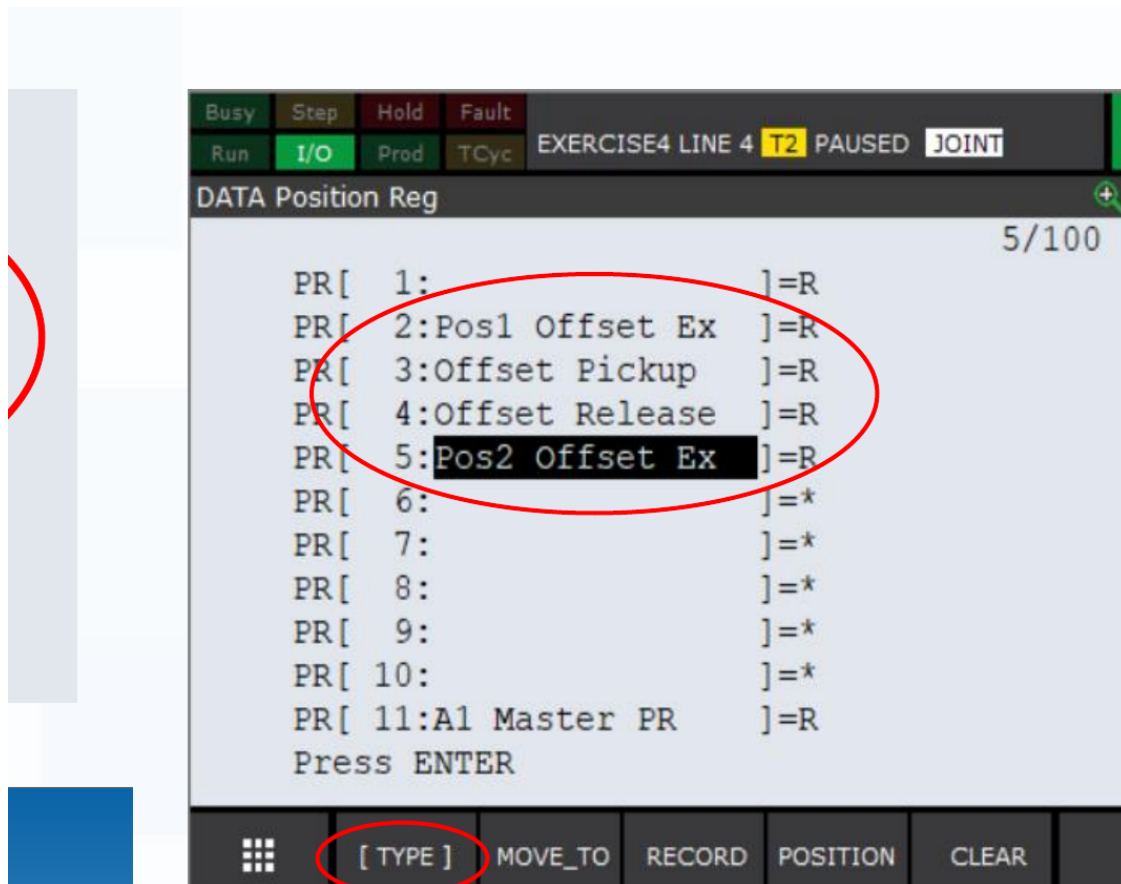


Figure 1: DATA $\square$ Position Reg screen. Use RECORD to store current pose, or POSITION to enter values manually.

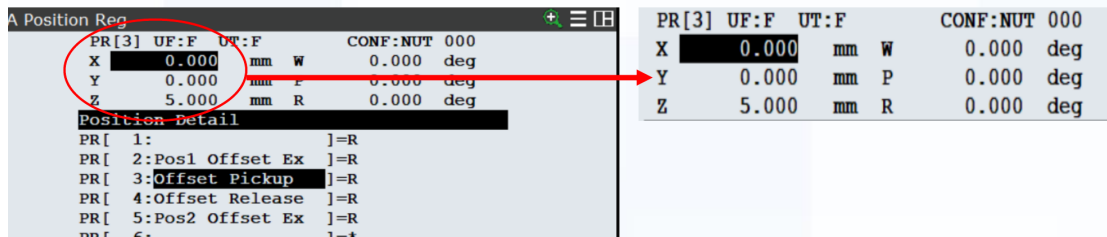


Figure 2: Entering offset values in a PR — set each component (X, Y, Z, W, P, R) individually.

Important:

- Double-check each angle value before pressing ENTER
- Units on FANUC are **degrees** - ensure you converted from radians
- Record the exact values you entered (rounding matters!)

2.2 2.2 Create a Test Program

Create a simple TP program to move to each position:

```
/PROG PYTHON_TEST
/MN
1: J PR[1] 10% FINE
2: PAUSE
3: J PR[2] 10% FINE
4: PAUSE
5: J PR[3] 10% FINE
6: PAUSE
7: J PR[4] 10% FINE
8: PAUSE
9: J PR[5] 10% FINE
10: PAUSE
/END
```

Why PAUSE? This lets you read the actual TCP position from the pendant display before moving to the next point.

2.3 2.3 Safety Check Before Running

Before executing:

- ☐ Verify all PR values are within joint limits
- ☐ Set speed override to **10%**
- ☐ Have the TA review your program
- ☐ Test first move (PR[1]) in **single-step mode**
- ☐ Keep hand on deadman switch at all times

3 Part 3: Measurement (25 minutes)

3.1 3.1 Run the Program and Record Positions

For each position:

1. Execute the move (robot moves to PR[n])
2. Wait for PAUSE
3. Read the **actual position** from the teach pendant:
 - Press **POSN** to see current position
 - Switch to **World** representation if needed
 - Record X, Y, Z values

3.2 3.2 Exercise 2: Complete the Measurement Table

Point	Actual X (mm)	Actual Y (mm)	Actual Z (mm)
1			
2			
3			
4			
5			

3.3 3.3 Exercise 3: Repeat Measurement (Repeatability Check)

Return to Point 1 and record the position again:

Trial	X (mm)	Y (mm)	Z (mm)
1st			
2nd			
3rd			

Calculate the spread (max - min) for each axis. This is a rough measure of repeatability.

4 Part 4: Error Analysis (20 minutes)

4.1 4.1 Compute Position Errors

Back in Python, calculate the error for each point:

```
# Enter your measured FANUC positions here (in mm)
fanuc_positions = [
    [0.0, 0.0, 0.0], # Point 1: Replace with actual values
    [0.0, 0.0, 0.0], # Point 2
    [0.0, 0.0, 0.0], # Point 3
    [0.0, 0.0, 0.0], # Point 4
    [0.0, 0.0, 0.0], # Point 5
]

print(f"{'Point':<6} {'Error X':>8} {'Error Y':>8} {'Error Z':>8} {'||Error||':>10}")
print("-" * 45)

for i, (result, actual) in enumerate(zip(results, fanuc_positions)):
    expected = result['pos_mm']
    actual = np.array(actual)
    error = actual - expected
    error_norm = np.linalg.norm(error)

    print(f"{i+1:<6} {error[0]:>8.2f} {error[1]:>8.2f} {error[2]:>8.2f} {error_norm:>10.2f}")
```

4.2 4.2 Exercise 4: Error Summary Table

Point	Error X (mm)	Error Y (mm)	Error Z (mm)	Total Error (mm)
1				
2				
3				
4				
5				
Mean				
Max				

4.3 4.3 Visualize the Errors

```
import matplotlib.pyplot as plt

# Bar chart of errors
points = [1, 2, 3, 4, 5]
errors = [...] # Fill in total errors

plt.figure(figsize=(8, 5))
plt.bar(points, errors, color='steelblue')
plt.xlabel('Point Number')
plt.ylabel('Position Error (mm)')
plt.title('Python vs FANUC Position Error')
plt.grid(axis='y', alpha=0.3)
```



```
plt.savefig('error_bar_chart.png', dpi=150)  
plt.show()
```

4.4 4.4 Exercise 5: Error Pattern Analysis

Answer these questions based on your measurements:

1. **Is the error consistent (systematic) or random?**

Hint: Do all errors point in roughly the same direction?

2. **Does error increase with distance from the robot base?**

Compare points close to the base vs far away.

3. **How does your measured error compare to the FANUC ER-4iA repeatability spec (± 0.02 mm)?**

Why is accuracy much worse than repeatability?

4. **What are the likely sources of the error you measured?**

Refer to lecture notes: geometric errors, compliance, backlash, thermal, servo.

5. **How could you reduce the error?**

List at least 2 methods discussed in the lecture.

5 Part 5: Improving the Match (10 minutes)

5.1 5.1 Quick Calibration Check

Try these corrections and see if the error decreases:

1. **Unit check:** Verify your DH parameters are in meters (not mm)
2. **Parameter check:** Compare your DH table against the official FANUC datasheet
3. **TCP offset:** If a tool is mounted, verify TCP calibration is current

5.2 5.2 Exercise 6: Discussion

Based on today's lab, answer:

1. **Why do industrial robots use taught positions instead of calculated positions for precision tasks?**
2. **When would offline programming (calculated positions) be preferred despite the accuracy limitation?**
3. **If you needed sub-millimeter accuracy from calculated positions, what would you do?**

6 Part 6: User Frame Portability (Extension)

Why This Matters

In industry, fixtures get repositioned regularly — a rotary table flips, a pallet moves between stations, a workpiece is replaced. If your program uses taught positions in the world frame, **every single point** must be re-taught. With a User Frame, you re-calibrate 3 points and everything updates automatically. This exercise lets you experience that difference firsthand.

6.1 6.1 Setup: Teach Positions in a User Frame

Exercise 7: Teach Points in a User Frame

1. Place a fixture (board, plate, or piece of paper) on the work surface
2. Calibrate **UFRAME[3]** on this fixture using the 3-point method:
 - Origin: one corner
 - +X direction: along one edge
 - +Y direction: along the perpendicular edge
3. Activate UFRAME[3] and UTOOL (from Lab 6)
4. Teach 3 positions on the fixture: **P[1]**, **P[2]**, **P[3]** (e.g., three holes or marked spots)
5. Write a simple test program:

```
/PROG UF_TEST
/MN
  1: UFRAME_NUM=3 ;
  2: UTOOL_NUM=2 ;
  3: L P[1:Spot_A] 200mm/sec FINE ;
  4: WAIT 1.00(sec) ;
  5: L P[2:Spot_B] 200mm/sec FINE ;
  6: WAIT 1.00(sec) ;
  7: L P[3:Spot_C] 200mm/sec FINE ;
  8: WAIT 1.00(sec) ;
/END
```

Run the program and verify the robot visits all 3 spots correctly.

Record the positions (read from pendant at each WAIT):

Point	X (mm)	Y (mm)	Z (mm)
P[1] — Position A			
P[2] — Position B			
P[3] — Position C			

6.2 6.2 Move the Fixture

Exercise 8: Move Fixture, Run Without Re-calibration

1. Physically move the fixture to a new position on the table (shift it 50–100 mm in any direction, or rotate it)
2. **Do NOT re-calibrate** the User Frame
3. Run UF_TEST again at **10% override, single-step mode**
4. Observe: does the robot still go to the correct spots on the fixture?

Caution

The robot will go to the **old** world-frame positions, which no longer correspond to the fixture. Move slowly and be ready to press E-Stop if the robot approaches the table surface.

Observation: Does the robot hit the correct spots on the moved fixture?

Why or why not?

6.3 6.3 Re-calibrate and Verify

Exercise 9: Re-calibrate User Frame

1. With the fixture in its **new** position, re-calibrate **UFRAME[3]** using the 3-point method (same procedure as before, but touching the new positions of the origin, X, and Y points)
2. Run UF_TEST again
3. Record the new positions from the pendant

After re-calibration:

Point	X (mm)	Y (mm)	Z (mm)	Matches fixture?
P[1] — Position A				
P[2] — Position B				
P[3] — Position C				

6.4 6.4 Analysis

1. How many points did you need to re-teach after moving the fixture?
2. If your program had 50 taught positions instead of 3, how much time would you save using User Frames vs re-teaching everything?
3. What would happen if you taught positions in UFRAME[3] but accidentally ran the program with UFRAME[1] active?

7 Summary

7.1 Key Takeaways

1. **Simulation and reality differ** — there is always an error gap between Python IK and the physical robot
2. **Accuracy vs Repeatability**: The FANUC is very repeatable but less accurate for calculated positions
3. **Error sources are multiple**: geometric, compliance, thermal, servo
4. **Calibration improves accuracy** but never eliminates all error
5. **User Frames make programs portable** — re-calibrate 3 points instead of re-teaching every position

Lab Completion:

- ☐ Python IK script completed with 5 target poses (Part 1)
- ☐ Joint angles transferred to FANUC PRs (Part 2)
- ☐ Actual TCP positions measured and recorded (Part 3)
- ☐ Error analysis completed with visualization (Parts 4–5)
- ☐ User Frame portability experiment completed (Part 6)

Instructor Signature: _____

Date: _____